Compiler Construction: Semantic Analysis Worksheet

Student Name and ID : _____
Date: _____

Objective: To understand the purpose, process, and implementation of the Semantic Analysis phase in a compiler. This phase ensures that the program has a valid meaning according to the language's rules.

Part 1: Core Concepts (Theory)

1. Multiple Choice & Short Answer

a) Semantic Analysis occurs **after** which phase and **before** which phase in the compiler?

- A) After Lexical Analysis, before Syntax Analysis
- B) After Syntax Analysis, before Code Generation
- C) After Code Generation, before Optimization
- D) After Optimization, before Linking

Answer: _____

b) What is the primary input to the Semantic Analyzer?

- A) Stream of characters
- B) Stream of tokens
- C) Abstract Syntax Tree (AST)
- D) Symbol Table

Answer: _____

c) The main data structure built and used during semantic analysis is the **_____**.

d) List three primary tasks of the Semantic Analysis phase:
1. _____
2. _____
3. _____

e) What is the key difference between a Syntax Error and a Semantic Error? Provide an example of each.

text

* **Syntax Error:** _____

  * **Example:** `if (x == 5 {` (Missing closing parenthesis)

* **Semantic Error:** _____

  * **Example:** _____

Part 2: The Symbol Table

2. Symbol Table Management

Imagine you are compiling the following code snippet. Your task is to manage the symbol table.

```
int main() {
   float radius = 5.5;
   float area;
   int count;
   area = 3.14159 * radius * radius;
   for (int i = 0; i < 10; i++) {
      count = count + 1;
   }
   return 0;
}
```

a) Fill in the symbol table below as the compiler would for the main function's scope. Include the **Name**, **Type**, and **Scope** for each variable.

Name	**Type**	**Scope**

b) The variable i inside the for loop has a limited scope. What happens to its entry in the symbol table when the semantic analyzer finishes processing the for loop?

Part 3: Type Checking in Action

3. Identify the Semantic Errors

For each of the following code snippets, identify the semantic error and state the type of error (e.g., "Type Mismatch", "Undeclared Variable", "Scope Violation").

a) **C-style Code:**
c int x = "hello"; // A string assigned to an integer
* **Error:** _____
* **Type of Error:** _____

b) **C-style Code:**
c { int y = 10; } y = y + 5; // Using 'y' outside its block
* **Error:** _____
* **Type of Error:** _____

c) **C-style Code:**
c int result; result = 5 + true; // Adding an integer and a boolean
* **Error:** _____
* **Type of Error:** _____

d) **C-style Code:**
c int foo() { // Function doesn't return a value }
* **Error:** _____
* **Type of Error:** _____

PART-4 : Syntax Directed Definition and Syntax Directed Translation

Match the following terms with their definitions:

Term	**Definition**
1. Syntax-Directed Definition (SDD)	A. Attributes whose value depends on parent or sibling nodes
2. Syntax-Directed Translation (SDT)	B. A context-free grammar with attributes and semantic rules
3. Synthesized Attribute	C. Program fragments embedded within grammar productions
4. Inherited Attribute	D. Attributes whose value depends only on children nodes

1.2 Short Answer Questions

a) What is the main difference between S-attributed and L-attributed SDDs?

_____

_____

b) Why can't inherited attributes be evaluated during LR parsing without modifications?

_____

_____

c) Give one advantage of using SDD/SDT over doing semantic analysis in a separate phase:

_____

S-Attributed Definitions (Bottom-Up Evaluation)

Grammar & Semantic Rules:

Production	**Semantic Rules**
1. $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
2. $E \rightarrow E_1 - T$	$E.val = E_1.val - T.val$
3. $E \rightarrow T$	$E.val = T.val$
4. $T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
5. $T \rightarrow T_1 / F$	$T.val = T_1.val / F.val$
6. $T \rightarrow F$	$T.val = F.val$
7. $F \rightarrow (E)$	$F.val = E.val$
8. $F \rightarrow digit$	$F.val = digit.lexval$

Problem: For the expression 3 * (2 + 4), show:

- The parse tree with attribute values
- The order of attribute computation

- **Pasrse Tree with Attributes**

- **The order of attribute computation**

1. _____ = _____

2. _____ = _____

3. _____ = _____

4. _____ = _____

5. _____ = _____

6. _____ = _____

7. _____ = _____

Compute the value for expression: (8 - 2) / 3 * 2

Show the parse tree and attribute computation steps:

Attribute Computation:

1. _____ = _____

2. _____ = _____

3. _____ = _____

4. _____ = _____

5. _____ = _____

6. _____ = _____

7. _____ = _____

8. _____ = _____

Final Value: _____

L-Attributed Definitions (Top-Down Evaluation)
3.1 Type Declaration Checking
Consider this SDD for variable declarations:
Grammar & Semantic Rules:

Production	**Semantic Rules**
1. D → T L	L.inh = T.type
2. T → int	T.type = integer
3. T → float	T.type = float
4. L → L_1 , id	L_1.inh = L.inh addType(id.entry, L.inh)
5. L → id	addType(id.entry, L.inh)

Problem: For the declaration float x, y, z, show:

- The parse tree with inherited and synthesized attributes
- The order of attribute computation

Parse Tree:

Attribute Flow:

1. _____ = _____

2. _____ = _____

3. _____ = _____

4. _____ = _____

5. _____ = _____

6. _____ = _____

7. _____ = _____

Trace the SDD for: int a, b, c, d

Show the parse tree and attribute computation:

Parse Tree:

Attribute Computation:

1. _____ = _____

2. _____ = _____

3. _____ = _____

4. _____ = _____

5. _____ = _____

6. _____ = _____

7. _____ = _____

Q1. Classify each attribute as synthesized or inherited:

Attribute	Definition
E.val	Value of an expression
decl.type	Type of a declaration
var.env	Environment (symbol table) passed down
L.inh	Inherited attribute for a list
S.code	Generated 3-address code

Q2. Mark which rules are legal for synthesized (S) and inherited (I):

- **A.val = B.val + C.val (S/I/Both)**
- **B.inh = A.inh + 1 (S/I/Both)**
- **S.code = generate(code) (S/I/Both)**

S-Attributed SDD (Expression Evaluation)

Grammar:

text

E → E1 + T { E.val = E1.val + T.val }

E → T { E.val = T.val }

T → T1 * F { T.val = T1.val * F.val }

T → F { T.val = F.val }

F → (E) { F.val = E.val }

F → num { F.val = num.lexval }

Q1. For input 3 + 4 * 5, annotate the parse tree with .val attributes.

Q2. Is this SDD S-attributed? Why?

Q3. Write the SDT (embedded actions) for an LR parser.

L-Attributed SDD (Declaration Type Checking)

Grammar:

text

D → T L

T → int { T.type = int }

T → float { T.type = float }

L → L1 , id { L1.inh = L.inh; id.type = L.inh; L.type = L1.type }

L → id { id.type = L.inh; L.type = L.inh }

Inherited attribute: L.inh carries type from T.

Q1. Show the annotated parse tree for: int a, b

Q2. Is this L-attributed? Can it be evaluated in a single left-to-right pass?

Q3. Convert this SDD into an SDT for a recursive descent parser.

Stack Implementation in SDT (Simulate Parser Stack)

Given grammar with embedded actions:

text

E → E1 + T { E.val = E1.val + T.val }

E → T

T → T1 * F { T.val = T1.val * F.val }

T → F

F → num { F.val = num.lexval }

Simulate stack for input 2 * 3 + 4

Show step-by-step:

- **Stack state (state, val, etc.)**
- **When num is shifted, its value is pushed.**
- **When reduction happens, compute attribute and push result.**

Step	Action	Stack (state, val)	Attribute computation
1	shift 2	[num(2)]	
...	...	...	
Final	reduce E	[E(10)]	

Semantic Rules for Syntax Trees

Goal: Build an AST (not evaluate).

Grammar with AST construction:

text

E → E1 + T { E.node = mkNode('+', E1.node, T.node) }

E → T { E.node = T.node }

T → T1 * F { T.node = mkNode('*', T1.node, F.node) }

T → F { T.node = F.node }

F → (E) { F.node = E.node }

F → num { F.node = mkLeaf(num.lexval) }

F → id { F.node = mkLeaf(id.name) }

Q1. Draw AST for a + b * c

Q2. Show the sequence of mkNode and mkLeaf calls during parsing.

Q3. Modify the SDD to also compute type (synthesized attribute .type) for each node.

S vs L-Attributed — Identify Type

For each SDD, state: S-attributed, L-attributed, or neither.

SDD	Rule
A	A.val = B.val + C.val B.inh = A.inh
B	S.val = A.val A.val = B.val + C.val
C	L.inh = T.type T.type = int
D	E.val = E1.val + T.val E.inh = 0 (not used in children)

Convert SDD to SDT with Explicit Stack

SDD:

text

S → A B C

A → a { A.val = 1 }

B → b { B.val = A.val + 1 } // depends on A

C → c { C.val = B.val + 1 }

S.val = C.val

Q1. Can this be evaluated in a single pass? Why?

Q2. Show the stack-based evaluation (assume bottom-up parser).
Hint: Inherited attributes need hack — use A.val stored after A reduced.

Q3. Rewrite as L-attributed SDD (if possible) by reordering grammar.

Realistic SDD for Tiny Language

Grammar for declarations and assignments:

text

P → D S

D → T id ; D | ε

T → int | float

S → id = E ; S | ε

E → E + T | T

T → id | num

Attributes needed:

- **.type for T, id, E**
- **.env (inherited) to store symbol table**
- **.code (synthesized) for intermediate code**

Q1. Write SDD rules to check id in assignment matches declared type.

Q2. Add inherited attribute D.env to pass symbol table down.

Q3. Mark which attributes are synthesized/inherited.